

Data & AI Engineer — Take-home Project

Late Deliveries & What Customers Say

Candidate: Nithyashree T

Project: Data & AI Engineer Take-home Project

Dataset: Brazilian E-Commerce Public Dataset by Olist

Tools: Python, Pandas, Jupyter Notebook, Power BI, ChatGPT, Claude, Git/GitHub

1. Project Overview

1.1 Problem Statement

The objective of this project is to understand how frequently customer orders are delivered late and whether late deliveries are associated with poorer customer satisfaction.

The analysis focuses on three main questions:

1. How many orders were delivered late?
2. Do customers give lower review scores when their orders are delivered late?
3. What do customers actually say about their experience in written reviews?

The project combines data engineering, business analytics, and AI-assisted review classification.

The analysis uses the Brazilian E-Commerce Public Dataset by Olist, which contains information about orders, order items, customer reviews, sellers, and other aspects of an e-commerce marketplace.

The project was divided into three required parts:

- **Part A:** Data Engineering
- **Part B:** Analytics using Power BI
- **Part C:** AI-based customer review classification

An additional **Data Platform** analysis was also completed as the optional extra.

2. Dataset and Tools

2.1 Dataset

The project uses the Brazilian E-Commerce Public Dataset by Olist from Kaggle.

Source:

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

The main datasets used were:

- `olist_orders_dataset.csv`
- `olist_order_items_dataset.csv`
- `olist_order_reviews_dataset.csv`

The optional Data Platform analysis additionally used:

- `olist_sellers_dataset.csv`

The raw dataset files were kept locally inside the `Dataset/` folder and were excluded from the GitHub repository using `.gitignore`.

2.2 Tools Used

Python and Pandas

Python with Pandas was used for:

- Loading CSV files
- Exploring the datasets
- Checking missing values
- Aggregating order-level information
- Joining datasets
- Calculating item counts and order values
- Calculating delivery delays
- Performing data-quality checks
- Preparing the final analytical dataset
- Preparing the AI classification data
- Performing the seller-level optional analysis

Jupyter Notebook

The Python implementation was developed in:

`Caspad_Data_AI_Project.ipynb`

The notebook contains the data preparation and validation workflow.

Power BI

Power BI Desktop was used for the required business analytics section.

The dashboard was created from the cleaned delivered-order dataset.

ChatGPT

ChatGPT was used for the Part C customer-review classification.

The same fixed prompt was used across batches of reviews.

Claude

Claude was used as an additional supporting tool for selected code reviews and comparison of implementation approaches.

Git and GitHub

Git was used for version control and GitHub was used to store and share the completed project.

3. Part A — Data Engineering

3.1 Loading the Orders Dataset

The first step was to load the Olist orders dataset using Pandas.

The orders dataset contained:

99,441 orders and 8 columns.

The columns included:

- `order_id`
- `customer_id`
- `order_status`
- `order_purchase_timestamp`
- `order_approved_at`
- `order_delivered_carrier_date`
- `order_delivered_customer_date`
- `order_estimated_delivery_date`

The `order_id` field was checked to ensure that each order was uniquely represented in the orders dataset.

The dataset contained several order statuses such as:

- delivered
- shipped
- canceled
- unavailable
- invoiced
- processing
- created
- approved

The project focuses on completed deliveries, so only orders with:

```
order_status = delivered
```

were retained for the final analysis.

This resulted in:

96,478 delivered orders.

Therefore:

2,963 non-delivered orders were excluded.

4. Exploring the Order Items Dataset

4.1 Loading Order Items

The order items dataset contained:

112,650 item records and 7 columns.

The columns included:

- `order_id`
- `order_item_id`
- `product_id`
- `seller_id`
- `shipping_limit_date`
- `price`
- `freight_value`

The dataset was checked for missing values.

No missing values were found in these columns.

A duplicate-row check was also performed.

There were no completely duplicated rows.

However, this does not mean that every `order_id` occurs only once.

An order can contain multiple products or multiple item rows.

5. Aggregating Order Items

Because the final analysis needs one row per order, the item-level data needed to be aggregated.

Two important order-level fields were created:

Item Count

The number of items in each order was calculated by grouping the item data by `order_id` and counting the item records.

This produced:

```
item_count
```

Order Value

The total order value was calculated by grouping the item data by `order_id` and summing the `price` column.

This produced:

```
order_value
```

Freight charges were not included in the order value because the project definition uses the sum of item prices.

The aggregation resulted in one row per order before joining the information back to the orders dataset.

6. Exploring Customer Reviews

The customer review dataset contained:

99,224 review records and 7 columns.

The columns included:

- `review_id`
- `order_id`
- `review_score`
- `review_comment_title`
- `review_comment_message`
- `review_creation_date`
- `review_answer_timestamp`

Many reviews did not contain written comments.

This is expected because a customer can provide a rating without writing a comment.

The review data also contained multiple review records for some orders.

Therefore, joining the raw review table directly to the order table could create duplicate order rows.

7. Creating an Order-Level Review Score

To avoid duplicate orders, the review data was first aggregated to the order level.

For orders with multiple review records, the mean review score was used.

The resulting field was:

```
review_score
```

This created one review-score value per order.

The review summary was then joined to the order-level item summary using a left join.

A left join was used so that orders without a review record would not be accidentally removed from the analysis.

8. Avoiding Duplicate Order Counting

One of the important data-engineering decisions in this project was to avoid row multiplication when joining datasets.

An order can have:

- multiple item rows
- multiple review rows

If the raw item table and raw review table were joined directly, one order could appear multiple times.

For example, if an order had three item rows and two review rows, a direct join could potentially produce six rows for that order.

To avoid this, both datasets were first reduced to one row per order:

Order Items → one row per order

Reviews → one row per order

Only after these aggregations were performed were they joined with the orders dataset.

This ensured that the final delivered-order table contained one row per order.

9. Filtering Delivered Orders

The combined order-level dataset was filtered using:

```
order_status = delivered
```

The resulting dataset contained:

96,478 delivered orders.

This became the main dataset used for the remaining analysis.

10. Calculating Delivery Delay

The project defines a late delivery as an order where the actual customer delivery date is later than the estimated delivery date.

Two date columns were converted to datetime format:

- `order_delivered_customer_date`
- `order_estimated_delivery_date`

The delivery difference was calculated as:

Actual delivery date – Estimated delivery date

The result can be:

- Negative → delivered early
- Zero → delivered on the estimated date
- Positive → delivered late

For the final `days_late` field, negative values were replaced with zero.

Therefore:

- Early delivery → `days_late = 0`
- On-time delivery → `days_late = 0`
- Late delivery → positive number of days

The calculation uses whole days.

11. Final Part A Dataset

The final delivered-order dataset contains:

96,478 rows and 12 columns.

Important fields include:

- `order_id`
- `customer_id`
- `order_status`
- `order_purchase_timestamp`
- `order_approved_at`
- `order_delivered_carrier_date`
- `order_delivered_customer_date`
- `order_estimated_delivery_date`
- `item_count`
- `order_value`
- `review_score`
- `days_late`

The final dataset was saved as:

`olist_delivered_orders.csv`

This file was then used for the Power BI analysis.

12. Data Quality Checks

Three important checks were performed as required by the project.

Check 1 — Order ID Uniqueness

The final delivered-order dataset was checked to ensure that `order_id` was unique.

Result: Passed

Each delivered order appeared exactly once.

Check 2 — Delivery Date vs Purchase Date

The delivered customer date was compared with the purchase timestamp.

The check looked for cases where:

Delivered Date < Purchase Date

Result: 0 records

This means no delivered order had a delivery date earlier than its purchase date.

Check 3 — Order Value Validation

The final `order_value` was compared with the raw sum of item prices from the original order-items dataset.

The difference was calculated for every delivered order.

Result: 0 mismatches

Therefore, the calculated order value matched the original item-level price data.

13. Missing Review Scores

The final delivered-order table contained:

646 delivered orders without a review score.

These values were not replaced with zero or another artificial value.

They were left as missing because the absence of a review should not be interpreted as a rating of zero.

These orders are automatically excluded when calculating average review scores.

Importantly:

- Missing `item_count`: 0
- Missing `order_value`: 0
- Missing `review_score`: 646

This distinction was preserved throughout the analysis.

14. Part B — Power BI Analytics

The cleaned file:

`olist_delivered_orders.csv`

was imported into Power BI Desktop.

The objective was to answer four analytical questions.

15. KPI 1 — Late Orders

A DAX measure was created to count orders where:

`days_late > 0`

The result was:

6,534 late orders

16. KPI 2 — Late Delivery Share

The late-order count was divided by the total number of delivered orders.

The result was:

6.77%

Therefore:

6.77% of delivered orders were delivered after the estimated delivery date.

17. Monthly Late-Delivery Analysis

A purchase-month field was created from the order purchase timestamp.

The month was stored in:

YYYY-MM

format.

A separate month-sort field was created to ensure that the months appeared chronologically rather than alphabetically.

A line chart was then created showing:

Late-Delivery Share by Purchase Month

The September 2016 point was excluded from the visual because it represented only one order and therefore produced a misleading 100% rate.

The monthly analysis was used to understand how late-delivery performance varied over time.

18. Average Review Score — Late vs On-Time

Orders were divided into two categories:

- On-Time
- Late

The average review score was calculated for each group.

Results

Delivery Status	Average Review Score
On-Time	4.29
Late	2.27

The difference was:

2.02 review-score points

Late deliveries had substantially lower average review scores than on-time deliveries.

This is an observed association in the dataset and should not be interpreted as proof that late delivery directly caused the lower scores.

19. Review Score for Orders More Than 7 Days Late

A separate Power BI measure was created for orders where:

```
days_late > 7
```

The average review score for these orders was:

1.70

This is lower than both:

- On-time orders: 4.29
- All late orders: 2.27

This suggests that severe delivery delays are associated with particularly poor customer feedback.

20. Power BI Dashboard

The Power BI dashboard was titled:

Olist Delivery Performance

The dashboard contains:

KPI Cards

- Late Orders: **6,534**
- Late Delivery Share: **6.77%**
- Average Review Score >7 Days Late: **1.70**

Monthly Trend

Late-Delivery Share by Purchase Month

Comparison Chart

Average Review Score: On-Time vs Late

Key Takeaway

Late deliveries represented 6.77% of delivered orders, but their average review score was substantially lower than on-time orders, falling further for orders that were more than seven days late.

21. Part C — AI Customer Review Analysis

The third part of the project focused on understanding what customers actually said in their written reviews.

The review dataset was filtered to include only reviews containing written comments.

This resulted in:

40,977 reviews with written comments.

A random sample of:

200 reviews

was selected using a fixed random seed.

This makes the selection reproducible.

22. Preparing the AI Review Sample

The selected reviews were joined with the delivered-order dataset using `order_id`.

This allowed each review to be associated with:

- `days_late`
- Delivery status

Each review was then classified as:

- Late
- On-Time

The 200-review sample contained:

- **21 Late reviews**
 - **179 On-Time reviews**
-

23. AI Classification Themes

Each review was assigned exactly one of five predefined themes.

Theme 1

`late_or_not_received`

Used for:

- Delivery delays
- Orders taking too long
- Orders not received
- Lost deliveries

Theme 2

`wrong_or_missing_item`

Used for:

- Wrong products
- Missing products
- Missing parts
- Incomplete orders

Theme 3

`product_quality`

Used for:

- Damaged products
- Broken products
- Defective products
- Poor product quality

Theme 4

`good_experience`

Used for:

- Clearly positive experiences
- Positive comments without a major complaint

Theme 5

other

Used for:

- Unclear comments
- Very short comments that did not fit another category
- Refund/payment/support issues that did not fit the defined categories
- Other unrelated cases

24. AI Prompt

The same fixed prompt was used for all 200 reviews.

The prompt instructed ChatGPT to:

- Choose exactly one theme.
- Select the most important theme when multiple issues were mentioned.
- Understand Portuguese reviews.
- Write summaries in English.
- Produce one short English summary per review.
- Avoid adding extra categories or explanations.

The exact prompt is saved separately as:

`part_c_ai_prompt.txt`

The raw model outputs are also saved as:

`part_c_ai_raw_outputs.txt`

This provides transparency and reproducibility.

25. AI Validation

To check whether the AI classifications were reliable, 30 reviews were manually examined.

The original review comments were read and independently assigned a theme.

The manual classification was compared with the ChatGPT classification.

Validation Result

30 / 30 classifications were correct

Validation accuracy: 100%

The validation results are saved as:

`part_c_manual_validation_30.csv`

This 100% result applies specifically to the manually checked sample of 30 reviews and should not be interpreted as guaranteed accuracy for every review.

26. AI Theme Results

The 200-review sample produced the following theme distribution.

Theme	Late	On-Time
Good experience	1 (4.8%)	113 (63.1%)
Late / not received	18 (85.7%)	10 (5.6%)
Other	0 (0.0%)	11 (6.1%)
Product quality	0 (0.0%)	27 (15.1%)
Wrong / missing item	2 (9.5%)	18 (10.1%)
Total	21 (100%)	179 (100%)

27. Main AI Finding

Within this 200-review sample:

85.7% of reviews from late orders were classified as delivery-related complaints.

For comparison:

5.6% of on-time reviews were classified as delivery-related complaints.

On-time reviews were predominantly positive:

63.1% were classified as good experiences.

The late group contains only 21 reviews, so these percentages should be treated as sample-level findings rather than estimates for the entire customer-review population.

28. Optional Data Platform Extra

The optional Data Platform track was also completed.

The Olist sellers dataset was loaded and connected with order-level delivery data.

The objective was to calculate late-delivery share at seller level.

29. Avoiding Duplicate Seller Orders

The order-items dataset can contain multiple item rows for the same order.

Therefore, counting item rows directly would overstate the number of orders handled by a seller.

To avoid this, unique:

```
order_id + seller_id
```

pairs were created first.

This ensured that an order containing multiple items from the same seller was counted only once for that seller.

30. Seller Eligibility

Only sellers with at least:

50 orders

were included in the optional analysis.

This resulted in:

429 eligible sellers.

The minimum seller order count was therefore 50.

Seller details were successfully matched for all eligible sellers.

31. Seller Late-Delivery Calculation

For each eligible seller, the following were calculated:

- Total orders
- Delivered orders
- Late orders
- Late-delivery share
- Seller city
- Seller state

The seller late-delivery share was calculated as:

Late delivered orders ÷ Delivered orders

The highest observed late-delivery share among eligible sellers was:

30.1%

This shows substantial variation in delivery performance across sellers.

The analysis can therefore help the Operations team prioritize sellers with consistently high late-delivery rates.

The seller-level dataset was saved as:

```
seller_late_share.csv
```

32. Overall Project Findings

The project produced several important findings.

Finding 1 — Late deliveries were not the majority

Out of:

96,478 delivered orders

there were:

6,534 late orders

representing:

6.77%

of delivered orders.

Finding 2 — Late orders had much lower review scores

Average review score:

- On-Time: **4.29**
- Late: **2.27**

This represents a difference of:

2.02 points

Finding 3 — Severe delays were associated with even lower ratings

Orders more than seven days late had an average review score of:

1.70

Finding 4 — Late-order reviews were strongly delivery-focused

Among the 21 late reviews in the 200-review sample:

85.7% were classified as delivery-related complaints.

Finding 5 — On-time reviews were predominantly positive

Among the 179 on-time reviews:

63.1% were classified as good experiences.

Finding 6 — Seller performance varied

Among 429 sellers with at least 50 orders, the highest observed seller late-delivery share was:

30.1%

This suggests that seller-level operational monitoring could help identify areas requiring attention.

33. Business Recommendations

Based on the results, three practical actions are recommended.

Recommendation 1 — Prioritize high-late-share sellers

The seller analysis showed significant variation in late-delivery performance.

The Operations team can prioritize sellers with consistently high late-delivery shares for investigation and follow-up.

The highest observed rate among eligible sellers was 30.1%.

The goal should be to identify whether these sellers have recurring operational or logistics issues.

Recommendation 2 — Intervene before the promised delivery date is missed

Late deliveries are associated with substantially lower review scores.

An operational monitoring process could identify orders that are at risk of missing their estimated delivery date.

Potential actions could include:

- Earlier escalation

- Logistics follow-up
- Delivery-status monitoring
- Proactive customer communication

The objective is to reduce the number of orders that cross the promised delivery date.

Recommendation 3 — Investigate severe delays separately

Orders more than seven days late had an average review score of only:

1.70

The Operations team should investigate the root causes of severe delays separately rather than treating all late deliveries as one group.

This could include analyzing:

- Seller
- Location
- Product category
- Logistics characteristics
- Delivery partners
- Order processing time

The current project identifies the problem; deeper operational data would be required to identify the exact causes.

34. Limitations

Sample Size for AI Analysis

The AI analysis used only 200 reviews.

Therefore, the theme percentages should not be treated as representative of all customer reviews.

Manual Validation Size

Only 30 reviews were manually validated.

The 100% accuracy result applies only to that validation sample.

Written Reviews Only

The AI analysis considered reviews with written comments.

Customers who provided only a numerical rating were not included in the AI classification.

Historical Dataset

The Olist dataset contains historical data from 2016–2018.

The findings may therefore not represent current e-commerce operations.

Association vs Causation

The analysis shows an association between delivery lateness and lower review scores.

It does not prove that delivery delay alone caused lower customer satisfaction.

Delivery Delay Granularity

`days_late` uses whole days.

Partial-day differences are therefore not represented.

35. What I Would Do Next

If more time and operational data were available, I would extend the project in several ways.

Larger AI Review Sample

Classify a larger number of customer reviews and increase the manual validation sample.

Automated AI Pipeline

Replace the manual batch process with an automated and repeatable classification pipeline.

Root-Cause Analysis

Analyze late deliveries by:

- Seller
- Product category
- Location
- Logistics information
- Delivery partner
- Order processing time

More Detailed Delay Analysis

Study whether customer satisfaction decreases progressively as the number of delayed days increases.

Monitoring

Create a recurring monitoring process for:

- Late-delivery share
- Severe delays
- Seller performance
- Customer satisfaction
- Delivery-related complaints

This would allow the Operations team to detect deteriorating performance earlier.

36. AI Usage and Transparency

AI tools were used selectively as supporting tools during the project.

The implementation, data analysis, validation, Power BI work, and final decisions were performed and verified as part of the project workflow.

ChatGPT

ChatGPT was used for:

- Occasional code review
- Troubleshooting
- Checking implementation approaches

- Part C customer-review classification

For Part C, the same fixed prompt was used to classify the selected 200 reviews.

Claude

Claude was used as an additional supporting tool for reviewing selected code and comparing implementation approaches.

Verification

AI-generated suggestions were reviewed and tested before being incorporated into the project.

Final:

- Calculations
- Data-quality checks
- Analytical results
- AI validation
- Power BI metrics
- Business recommendations

were verified against the project data.

The exact Part C prompt and raw AI outputs were saved in the project files.

37. Project Outputs

The completed project contains the following main files:

Python Notebook

`Caspad_Data_AI_Project.ipynb`

Contains the data preparation, validation, AI preparation, and optional seller analysis.

Cleaned Dataset

`olist_delivered_orders.csv`

Contains the final one-row-per-delivered-order dataset used for Power BI.

Power BI Dashboard

`Analysis.pbix`

Contains the required delivery-performance dashboard.

AI Results

`part_c_ai_results.csv`

Contains the structured AI classifications and summaries for the 200 reviews.

AI Prompt

`part_c_ai_prompt.txt`

Contains the exact prompt used for the Part C classification.

Raw AI Outputs

`part_c_ai_raw_outputs.txt`

Contains the raw ChatGPT classification outputs.

Manual Validation

`part_c_manual_validation_30.csv`

Contains the manual validation results for 30 reviews.

Seller Analysis

`seller_late_share.csv`

Contains the optional seller-level late-delivery analysis.

README

`readme.md`

Contains the required project documentation, assumptions, methodology, results, limitations, AI usage, and optional extra.

38. GitHub and Reproducibility

The project was version-controlled using Git and uploaded to GitHub.

GitHub repository:

<https://github.com/nithyashree3035/caspad-data-ai-engineer-takehome>

The raw Olist `Dataset/` folder was excluded from the repository using `.gitignore`.

This prevents the original raw dataset files from being uploaded while still allowing another user to download the dataset from the original Kaggle source and reproduce the analysis.

The project was committed with Git and the `main` branch was pushed to GitHub.

39. Final Project Summary

This project combined data engineering, business analytics, and AI-assisted text analysis to investigate late deliveries and customer experience.

The data-engineering stage produced a reliable order-level dataset containing **96,478 delivered orders**.

The Power BI analysis showed:

- **6,534 late orders**
- **6.77% late-delivery share**
- **4.29 average review score for on-time orders**
- **2.27 average review score for late orders**
- **1.70 average review score for orders more than seven days late**

The AI analysis of 200 written reviews found that:

- **85.7% of late-order reviews were delivery-related complaints**
- **63.1% of on-time reviews were good experiences**

The optional seller analysis found:

- **429 sellers** with at least 50 orders
- Highest observed seller late-delivery share of **30.1%**

The project demonstrates a complete workflow from raw e-commerce data through data cleaning, validation, business analytics, AI-assisted customer feedback analysis, and operational recommendations.

The main lesson from the project is that a relatively small share of orders were late, but those late deliveries were associated with substantially poorer customer feedback, making delivery reliability an important operational area to monitor.